Übungsskript: Systematische Datenanalyse mit Python

Modul: Coding und Digitalisierung

Grundlagen bis erweiterte Techniken

Wintersemester 2026, P. Kern

Inhaltsverzeichnis

1 Einführung und organisatorische Hinweise **3**
- 1.1 Verwendund von KI . . . 3
- 1.2 Das Projekt: Smart City Sensornetzwerk . . . 4
- 1.3 Lernziele . . . 4

2 Modul 1: Vorbereitung und Setup **5**
- 2.1 Aufgabe 1.1: Ordnerstruktur anlegen . . . 5
- 2.2 Aufgabe 1.2: Virtuelle Umgebung (venv) erstellen . . . 5
- 2.3 Aufgabe 1.3: Der erste Test . . . 5
- 2.4 Kontrollfragen zu Modul 1 . . . 5

3 Modul 2: Variablen und Datentypen **6**
- 3.1 Aufgabe 2.1: Sensordaten simulieren . . . 6
- 3.2 Aufgabe 2.2: Typen prüfen und ausgeben . . . 6
- 3.3 Aufgabe 2.3: Mathematische Operatoren anwenden . . . 6
- 3.4 Kontrollfragen zu Modul 2 . . . 6

4 Modul 3: Kontrollstrukturen (If-Abfragen) **7**
- 4.1 Aufgabe 3.1: Sensorstatus prüfen . . . 7
- 4.2 Aufgabe 3.2: Mehrfache Verzweigungen (elif) . . . 7
- 4.3 Aufgabe 3.3: Logische Operatoren (and / or) . . . 7
- 4.4 Kontrollfragen zu Modul 3 . . . 7

5 Modul 4: Schleifen (Loops) **8**
- 5.1 Aufgabe 4.1: Die while-Schleife (Simulation eines Heizprozesses) . . . 8
- 5.2 Aufgabe 4.2: Die for-Schleife mit range() . . . 8
- 5.3 Aufgabe 4.3: Schleifensteuerung (break und continue) . . . 8
- 5.4 Kontrollfragen zu Modul 4 . . . 8

6 Modul 5: Datenstrukturen - Listen und Dictionaries **9**
- 6.1 Aufgabe 5.1: Listen erstellen und manipulieren . . . 9
- 6.2 Aufgabe 5.2: Über Listen iterieren . . . 9
- 6.3 Aufgabe 5.3: Dictionaries . . . 9
- 6.4 Aufgabe 5.4: Liste von Dictionaries . . . 9
- 6.5 Kontrollfragen zu Modul 5 . . . 9

7 Modul 6: CSV Dateien einlesen und Klassische Analyse **10**
7.1 Aufgabe 6.1: Datei einlesen . . . 10
7.2 Aufgabe 6.2: Daten parsen und Ausreißer filtern . . . 10
7.3 Aufgabe 6.3: Mathematische Auswertung (Standard-Python) . . . 11
7.4 Kontrollfragen zu Modul 6 . . . 11

8 Modul 7: Einführung in NumPy **12**
8.1 Aufgabe 7.1: Installation und Import . . . 12
8.2 Aufgabe 7.2: Das NumPy Array erstellen . . . 12
8.3 Aufgabe 7.3: Vektorisierte Mathematik . . . 12
8.4 Kontrollfragen zu Modul 7 . . . 12

9 Modul 8: Fortgeschrittene NumPy Analyse (Boolesche Maskierung) **13**
9.1 Aufgabe 8.1: Daten laden direkt mit NumPy . . . 13
9.2 Aufgabe 8.2: Boolesche Maskierung anwenden . . . 13
9.3 Aufgabe 8.3: Statistische Kennzahlen direkt abrufen . . . 13
9.4 Kontrollfragen zu Modul 8 . . . 13

10 Modul 9: Visualisierung der Daten mit Matplotlib **14**
10.1 Aufgabe 9.1: Matplotlib installieren und importieren . . . 14
10.2 Aufgabe 9.2: Ein einfaches Liniendiagramm . . . 14
10.3 Aufgabe 9.3: Subplots (Vorher / Nachher Vergleich) . . . 14
10.4 Kontrollfragen zu Modul 9 . . . 14

11 Modul 10: Erweiterte Visualisierung mit Seaborn **15**
11.1 Aufgabe 10.1: Seaborn installieren . . . 15
11.2 Aufgabe 10.2: Der Boxplot zur Ausreißererkennung . . . 15
11.3 Aufgabe 10.3: Der finale Report (Kombination aller Techniken) . . . 15
11.4 Kontrollfragen zu Modul 10 . . . 15

1 Einführung und organisatorische Hinweise

Willkommen zum Übungsskript des Moduls „Coding und Digitalisierung“. In diesem Skript werden Sie das theoretische Wissen aus den Vorlesungen praktisch anwenden. Das Skript ist als zusammenhängendes Projekt konzipiert, das Sie über mehrere Stunden hinweg bearbeiten können. Sie starten mit einfachen Variablen und arbeiten sich über Schleifen, Kontrollstrukturen und mathematische Operationen bis hin zu den Industriestandards der Datenanalyse (NumPy) und Visualisierung (Matplotlib, Seaborn) vor.

1.1 Verwendund von KI

Sie können die KI der Technischen Hochschule Köln **kostenfrei** für Ihre Fragen nutzen: https://chat.kiconnect.nrw/app

Wichtig: Fragen Sie **nicht** nach der reinen Lösung, sondern immer nach einer ausführlichen **Erklärung**. Eine bloße Lösung unterstützt das Lernen kaum – erst die nachvollziehbare Erklärung fördert das Verständnis.

Beispiel aus 3.2:
Frage: Was sind *f-strings* und wozu kann ich diese nutzen?

Antwort:

f-strings (format-strings) wurden in Python 3.6 eingeführt und ermöglichen das Einbetten von Python-Ausdrücken direkt in Zeichenketten. Der Präfix `f` (bzw. `F`) vor dem String-Literal signalisiert, dass alles, was in geschweiften Klammern `{}` steht, ausgewertet und in den resultierenden Text eingefügt wird.

Vorteile

- **Lesbarkeit:** Der Code ist kompakt und leicht zu verstehen.
- **Performance:** f-strings sind meist schneller als die klassische %-Formatierung oder `str.format()`.
- **Flexibilität:** Jeder Python-Ausdruck, Format-Spec (z. B. `:.2f`) und Funktionsaufruf kann verwendet werden.

Beispiel

```
name  = "Anna"
punkte = 9.75
print(f"Hallo {name}, du hast {punkte:.1f} Punkte erreicht.")
# → Hallo Anna, du hast 9.8 Punkte erreicht.
```

1.2 Das Projekt: Smart City Sensornetzwerk

Sie arbeiten als Datenanalyst für ein Smart City Projekt. In der Stadt sind verschiedene Sensoren verteilt, die Temperatur und Luftfeuchtigkeit messen. Leider gibt es Störungen im Netzwerk: Einige Sensoren liefern unplausible Werte (Ausreißer), andere sind zeitweise inaktiv.

Ihre Aufgabe ist es, ein Python-Skript (z.B. `main.py`) zu entwickeln, das diese Daten einliest, bereinigt, mathematisch auswertet und am Ende professionell visualisiert.

1.3 Lernziele

- Einrichtung einer professionellen Entwicklungsumgebung.
- Verständnis von dynamischer Typisierung und der Python Virtual Machine (PVM).
- Beherrschung von Kontrollstrukturen (If-Abfragen, Schleifen).
- Umgang mit elementaren Datenstrukturen (Listen, Dictionaries).
- Algorithmisches Denken (Filterung, Aggregation ohne Bibliotheken).
- Vektorisiertes Rechnen mit NumPy.
- Datenvisualisierung mit Matplotlib und Seaborn.

2 Modul 1: Vorbereitung und Setup

Bevor wir mit dem Code beginnen, müssen wir unsere Arbeitsumgebung richtig einrichten. Python ist nicht nur eine Sprache, sondern ein ganzes Ökosystem.

2.1 Aufgabe 1.1: Ordnerstruktur anlegen

Erstellen Sie auf Ihrem Computer einen neuen Ordner namens `SmartCityProject`. Öffnen Sie diesen Ordner in Visual Studio Code (VS Code). Erstellen Sie in diesem Ordner eine Datei mit dem Namen `main.py` und eine Datei namens `messdaten.csv`. Kopieren Sie die bereitgestellten CSV-Daten in diese Datei.

2.2 Aufgabe 1.2: Virtuelle Umgebung (venv) erstellen

In der Vorlesung haben Sie gelernt, dass eine virtuelle Umgebung (venv) dazu dient, Pakete isoliert von anderen Projekten zu verwalten. Das verhindert Versionskonflikte, wenn Projekt A eine ältere Version einer Bibliothek benötigt als Projekt B.

Tipp zur Einrichtung eines venv:
Öffnen Sie das Terminal in VS Code. Geben Sie folgenden Befehl ein:
Für Windows: `python -m venv .venv`
Für Mac/Linux: `python3 -m venv .venv`
Aktivieren Sie das Environment anschließend. VS Code fragt Sie oft direkt unten rechts, ob diese neue Umgebung verwendet werden soll. Klicken Sie auf „Ja“.
Sie können das venv auch über das Terminal aktieren.
Für Windows: `.\.venv\Scripts\activate`
Für Mac/Linux: `source .venv/bin/activate`

2.3 Aufgabe 1.3: Der erste Test

Schreiben Sie in Ihre `main.py` folgenden Code, um zu testen, ob Python korrekt funktioniert. In der Vorlesung haben Sie gelernt, dass Python auf extrem leicht lesbaren Code fokussiert ist (inspiriert von Monty Python's Flying Circus).

```
1 print("Systemstart: Smart City Analyse")
```

Führen Sie das Skript aus.
Erwartetes Ergebnis im Terminal:

```
Systemstart: Smart City Analyse
```

2.4 Kontrollfragen zu Modul 1

Beantworten Sie für sich folgende Fragen (schreiben Sie die Antworten als Kommentar `#` in Ihr Skript):

1. Wer hat Python erfunden und in welchem Jahr wurde es veröffentlicht?
2. Was passiert im Hintergrund, wenn Sie eine `.py` Datei ausführen? Erklären Sie die Begriffe *Bytecode* und *Python Virtual Machine (PVM)*.
3. Warum ist es sinnvoll, in jedem neuen Projekt eine virtuelle Umgebung (`venv`) anzulegen?

3 Modul 2: Variablen und Datentypen

Python nutzt *dynamische Typisierung*. Das bedeutet, Sie müssen nicht deklarieren, ob eine Variable ein Text (String) oder eine Zahl (Integer) ist. Python erkennt dies zur Laufzeit automatisch. In unserer CSV-Datei kommen vier grundlegende Datentypen vor.

3.1 Aufgabe 2.1: Sensordaten simulieren

Legen Sie in Ihrem Skript vier Variablen an, die eine einzelne Zeile aus unserer CSV-Datei repräsentieren. Nutzen Sie dafür die passenden Datentypen.

- `sensor_id`: Die Zahl 1
- `standort`: Der Text "Nordpark"
- `temperatur`: Die Zahl 22.5
- `ist_aktiv`: Der Wahrheitswert Wahr

3.2 Aufgabe 2.2: Typen prüfen und ausgeben

Nutzen Sie die eingebaute Python-Funktion `type()`, um den Datentyp jeder Variablen zu ermitteln. Geben Sie das Ergebnis mit `print()` aus. Nutzen Sie sogenannte f-Strings, um Variablen in einen Text einzubetten.

Erwartetes Ergebnis im Terminal:

```
Datentyp von ID: <class 'int'>
Datentyp von Standort: <class 'str'>
Datentyp von Temperatur: <class 'float'>
Datentyp von Aktiv-Status: <class 'bool'>
```

3.3 Aufgabe 2.3: Mathematische Operatoren anwenden

Wir müssen die Temperatur in Fahrenheit umrechnen. Die mathematische Formel lautet:

$$Fahrenheit = Temperatur \times 1.8 + 32$$

Berechnen Sie den Wert und speichern Sie ihn in einer neuen Variablen `temp_fahrenheit`. Geben Sie aus: „22.5 Grad Celsius entsprechen [X] Grad Fahrenheit."

Erwartetes Ergebnis im Terminal:

```
22.5 Grad Celsius entsprechen 72.5 Grad Fahrenheit.
```

3.4 Kontrollfragen zu Modul 2

1. Was ist der Unterschied zwischen `int` und `float`?
2. Was bedeutet dynamische Typisierung konkret für Sie als Programmierer im Vergleich zu Sprachen wie Java oder C++?
3. Welcher Operator wird in Python für die Potenzierung (z.B. 2 hoch 3) verwendet? (Tipp: siehe Vorlesungsfolien zu Operatoren).

4 Modul 3: Kontrollstrukturen (If-Abfragen)

In diesem Modul lernen Sie Verzweigungen kennen. Ein intelligentes System muss auf verschiedene Sensorwerte unterschiedlich reagieren. Wie in der Vorlesung besprochen, nutzt Python Einrückungen (meist 4 Leerzeichen), um Code-Blöcke zu definieren. Es gibt keine geschweiften Klammern `{}`.

4.1 Aufgabe 3.1: Sensorstatus prüfen

Schreiben Sie eine einfache `if`-Abfrage, die prüft, ob die Variable `ist_aktiv` den Wert `True` hat. Wenn ja, geben Sie „Sensor sendet Daten“ aus. Fügen Sie einen `else`-Block hinzu, der „Sensor ist offline! “ ausgibt, falls der Wert `False` ist.

Ändern Sie testweise den Wert der Variable `ist_aktiv` auf `False`, um beide Pfade zu testen.

4.2 Aufgabe 3.2: Mehrfache Verzweigungen (elif)

Wir wollen die Temperatur in verschiedene Kategorien einteilen, um Warnungen auszugeben. Schreiben Sie eine erweiterte Kontrollstruktur (`if`, `elif`, `else`), die folgende Bedingungen für die Variable `temperatur` prüft:

- Ist die Temperatur größer als 50, geben Sie „WARNUNG: Hitze-Ausreißer erkannt (Möglicher Defekt)! “ aus.
- Ist die Temperatur kleiner als 0, geben Sie „WARNUNG: Frost-Ausreißer erkannt! “ aus.
- Ist die Temperatur zwischen 0 und 30 (inklusive), geben Sie „Temperatur im Normalbereich.“ aus.
- In allen anderen Fällen geben Sie „Erhöhte Temperatur, aber im Rahmen.“ aus.

Testen Sie Ihren Code, indem Sie die `temperatur` Variable auf 105.0 setzen.
Erwartetes Ergebnis im Terminal (bei 105.0):

```
WARNUNG: Hitze-Ausreißer erkannt (Möglicher Defekt)!
```

4.3 Aufgabe 3.3: Logische Operatoren (and / or)

Verknüpfen Sie Bedingungen. Es soll nur dann eine Warnung ausgegeben werden, wenn die Temperatur über 50 liegt **und** der Sensor aktiv ist. Nutzen Sie das Schlüsselwort `and`.

Tipp zu logischen Operatoren:
In Python verwenden wir die englischen Wörter `and`, `or`, `not`, um Wahrheitswerte zu verknüpfen. Zum Beispiel: `if temperatur > 50 and ist_aktiv == True:`

4.4 Kontrollfragen zu Modul 3

1. Warum führt eine fehlende Einrückung in Python nach einem `if`-Statement zu einem Fehler (IndentationError)?
2. Was bewirkt das Schlüsselwort `elif` und wann verwendet man es anstelle von mehreren einzelnen `if`-Statements?
3. Welche Vergleichsoperatoren kennen Sie? Zählen Sie mindestens vier auf (z.B. ==).

5 Modul 4: Schleifen (Loops)

Oft müssen wir Vorgänge wiederholen. In diesem Modul lernen wir die `while`- und `for`-Schleifen kennen.

5.1 Aufgabe 4.1: Die while-Schleife (Simulation eines Heizprozesses)

Stellen Sie sich vor, der Sensor misst einen Raum, der hochgeheizt wird. Erstellen Sie eine Variable `raum_temp = 15.0`. Schreiben Sie eine `while`-Schleife, die solange läuft, wie die `raum_temp` kleiner als 22.0 ist. In jedem Schleifendurchlauf:

- Geben Sie die aktuelle Temperatur aus.
- Erhöhen Sie die `raum_temp` um 2.5 Grad.

Erwartetes Ergebnis im Terminal:

```
Heize... aktuelle Temp: 15.0
Heize... aktuelle Temp: 17.5
Heize... aktuelle Temp: 20.0
Zieltemperatur erreicht.
```

5.2 Aufgabe 4.2: Die for-Schleife mit range()

Die `for`-Schleife ist ideal, um eine festgelegte Anzahl von Wiederholungen durchzuführen. Nutzen Sie `for i in range(...):` um die Zahlen von 1 bis 5 auszugeben. Geben Sie dabei das Quadrat der jeweiligen Zahl aus (nutzen Sie den Potenz-Operator `**`).

Erwartetes Ergebnis im Terminal:

```
Sensor-Kalibrierungsschritt 1: 1
Sensor-Kalibrierungsschritt 2: 4
Sensor-Kalibrierungsschritt 3: 9
Sensor-Kalibrierungsschritt 4: 16
Sensor-Kalibrierungsschritt 5: 25
```

5.3 Aufgabe 4.3: Schleifensteuerung (break und continue)

Kopieren Sie Ihre Schleife aus 4.2. Fügen Sie eine `if`-Abfrage ein: Wenn die Zahl `i` genau 3 ist, soll dieser Durchlauf übersprungen werden (ohne die Rechnung auszuführen). Welches Schlüsselwort aus der Vorlesung benötigen Sie dafür? (`break` oder `continue`?). Wenn die Zahl 5 erreicht wird, soll die Schleife sofort und komplett abgebrochen werden.

Hinweis: break und continue wurde nicht in der VL angeprochen. Erarbeiten Sie es selbst.

5.4 Kontrollfragen zu Modul 4

1. Was ist der konzeptionelle Unterschied zwischen einer `for`-Schleife und einer `while`-Schleife?
2. Erklären Sie mit eigenen Worten den Unterschied zwischen den Schlüsselwörtern `break` und `continue`.
3. Wie rufen Sie die `range` Funktion auf, wenn Sie bei 0 starten, bis 10 gehen und in 2er-Schritten zählen wollen?

6 Modul 5: Datenstrukturen - Listen und Dictionaries

Bisher haben wir nur mit einzelnen Werten gearbeitet. Ein Datensatz besteht jedoch aus vielen Werten. Hier kommen Listen und Dictionaries ins Spiel.

6.1 Aufgabe 5.1: Listen erstellen und manipulieren

Erstellen Sie eine Liste namens `messwerte_liste`, die folgende Temperaturen enthält: 22.5, 23.1, 105.0, 21.8, -5.0, 24.0, 110.5.

- Fügen Sie einen neuen Messwert `23.0` am Ende der Liste hinzu (Tipp: `.append()`).
- Der Wert -5.0 ist ein bekannter Messfehler aufgrund eines Kabelbruchs. Entfernen Sie ihn aus der Liste (Tipp: `.remove()`).
- Geben Sie die Länge der Liste mithilfe der Funktion `len()` aus.

Erwartetes Ergebnis im Terminal:

```
Aktualisierte Liste: [22.5, 23.1, 105.0, 21.8, 24.0, 110.5, 23.0]
Anzahl der Messwerte: 7
```

6.2 Aufgabe 5.2: Über Listen iterieren

Schreiben Sie eine `for`-Schleife, die über alle Werte in `messwerte_liste` iteriert. Addieren Sie alle Werte auf eine Variable `gesamtsumme` (die Sie vor der Schleife mit 0 initialisieren). Berechnen Sie danach den Durchschnitt (Summe geteilt durch Anzahl der Elemente) und geben Sie ihn aus.

6.3 Aufgabe 5.3: Dictionaries

Während Listen gut für Folgen von Werten sind, eignen sich Dictionaries perfekt, um strukturierte Daten mit Eigenschaften zu speichern. Erstellen Sie ein Dictionary `sensor_dict`, das alle Informationen eines einzelnen Sensors speichert (Nutzen Sie die Struktur aus der CSV: ID, Standort, Temperatur, IstAktiv). Greifen Sie anschließend explizit auf den Standort zu und drucken Sie ihn aus.

Erwartetes Ergebnis im Terminal:

```
Der analysierte Sensor befindet sich am Standort: Nordpark
```

6.4 Aufgabe 5.4: Liste von Dictionaries

Kombinieren Sie beide Konzepte. Eine Liste kann verschiende Elemente enthalten, auch Dictonaries. Erstellen Sie eine Liste, die zwei Sensor-Dictionaries enthält. Iterieren Sie mit einer `for`-Schleife über die Liste und geben Sie für jeden Sensor den Satz aus: „Sensor [ID] am Standort [Standort] hat eine Temperatur von [Temperatur]“.

6.5 Kontrollfragen zu Modul 5

1. Erklären Sie den Unterschied zwischen einer Liste (`[ ]`) und einem Tupel (`( )`). (Tipp: mutable vs. immutable).
2. Wann würden Sie ein Dictionary einer Liste vorziehen?
3. In der Vorlesung wurde erwähnt, dass reine Python-Listen bei Mathematik langsam sind. Welches Modul wird später eingeführt, um dieses Problem zu lösen?

7 Modul 6: CSV Dateien einlesen und Klassische Analyse

Nun lesen wir echte Daten ein und wenden das an, was im Dokument `002_Beispiel Datenstrukturen - Analyse.pdf` vorgestellt wurde: Die klassische, nicht-vektorisierte Datenanalyse mit Schleifen.

7.1 Aufgabe 6.1: Datei einlesen

Um in Python Textdateien (wie eine `.csv`) einzulesen, benötigen wir zunächst keine externen Bibliotheken. Wir verwenden stattdessen die integrierte `open()`-Funktion.

Es ist wichtig, dass einmal geöffnete Dateien auch wieder vom Betriebssystem geschlossen werden, um Arbeitsspeicher freizugeben und Dateisperren (Locks) zu vermeiden. In Python übernimmt der sogenannte `with`-Block diese Aufgabe automatisch für uns.

Schauen Sie sich das folgende Code-Beispiel an:

```
with open('sensor_messdaten_projekt.csv', 'r') as file:
    # Alles, was hier eingerückt ist, passiert solange die Datei offen ist.
    zeilen = file.readlines()

# Hier, ohne Einrückung, hat Python die Datei bereits automatisch geschlossen.
```

Ihre Aufgabe: Nutzen Sie den obigen Code als Vorlage, um die Datei `sensor_messdaten_projekt.csv` einzulesen. Ermitteln Sie anschließend (außerhalb des `with`-Blocks) mit der Funktion `len()`, wie viele Zeilen sich in der Liste `zeilen` befinden und lassen Sie sich diesen Wert mit einem erklärenden Text per `print()` ausgeben.

Tipp zur Dateiverarbeitung:
Der `with`-Block stellt sicher, dass die Datei nach dem Einlesen *immer* automatisch wieder geschlossen wird, selbst wenn im eingerückten Code ein Fehler auftritt. Dies ist Best Practice in Python.
Das `'r'` in `open(..., 'r')` steht übrigens für *read* (lesen). Würden Sie Daten in eine Datei schreiben wollen, müssten Sie den Modus `'w'` (*write*) oder `'a'` (*append*) wählen.

7.2 Aufgabe 6.2: Daten parsen und Ausreißer filtern

Extrahieren Sie aus den eingelesenen Zeilen ausschließlich die Spalte „Temperatur“. Überspringen Sie die erste Zeile (den Header). Erstellen Sie zwei leere Listen: `gute_daten` und `ausreisser`. Gehen Sie in einer Schleife durch die extrahierten Temperaturen (wandeln Sie diese mit `float()` um).

- Liegt der Wert zwischen 0 und 50, fügen Sie ihn zu `gute_daten` hinzu.
- Liegt der Wert darüber oder darunter, fügen Sie ihn zu `ausreisser` hinzu.

Erwartetes Ergebnis im Terminal:

```
Klassische Analyse gestartet...
Gefundene valide Datenpunkte: 11
Gefundene Ausreißer: 3
```

7.3 Aufgabe 6.3: Mathematische Auswertung (Standard-Python)

Berechnen Sie nun ausschließlich aus der Liste `gute_daten`:

1. Den Mittelwert (Mean).
2. Das Minimum und das Maximum.

Vermeiden Sie hierbei externe Bibliotheken, nutzen Sie Schleifen und Standard-Funktionen wie in Listing 2 der Vorlesung gezeigt.

7.4 Kontrollfragen zu Modul 6

1. Warum müssen eingelesene Daten aus einer Textdatei (CSV) explizit mit `float()` oder `int()` umgewandelt werden, bevor man damit rechnen kann?
2. Beschreiben Sie den algorithmischen Aufwand: Warum ist die Datenbereinigung mittels `for`-Schleifen bei extrem großen Datensätzen (z.B. Millionen von Zeilen) ineffizient?

8 Modul 7: Einführung in NumPy

Die klassische Analyse mit Schleifen funktioniert, ist aber langsam und der Code wird schnell unübersichtlich. Wie in Vorlesung 002 vorgestellt, ist `NumPy` der Industriestandard für wissenschaftliches Rechnen. Es basiert auf in C geschriebenen N-dimensionalen Arrays und ersetzt ineffiziente for-Schleifen durch vektorisierte Operationen.

8.1 Aufgabe 7.1: Installation und Import

Um NumPy nutzen zu können, müssen wir es in unsere virtuelle Umgebung installieren.

Tipp zur Installation von Modulen:
Öffnen Sie Ihr VS Code Terminal (achten Sie darauf, dass das `venv` aktiv ist, erkennbar an einem `(.venv)` vor dem Pfad). Geben Sie ein:
`pip install numpy`
Importieren Sie es anschließend oben in Ihrem `main.py` Skript mit: `import numpy as np`

8.2 Aufgabe 7.2: Das NumPy Array erstellen

Nehmen Sie Ihre bereinigte Liste `gute_daten` aus Aufgabe 6 und wandeln Sie diese in ein NumPy-Array um: `daten_np = np.array(gute_daten)`

Überprüfen Sie den Datentyp der neuen Variable mit `type()`.

Erwartetes Ergebnis im Terminal:

```
Typ des NumPy Objekts: <class 'numpy.ndarray'>
```

8.3 Aufgabe 7.3: Vektorisierte Mathematik

Erinnern Sie sich an Aufgabe 2.3 (Celsius zu Fahrenheit)? Damals haben wir einen Wert umgerechnet. Bei einer normalen Liste bräuchten wir jetzt eine Schleife. Mit NumPy geht das ohne Schleife für alle Werte gleichzeitig!

Führen Sie die Rechnung `fahrenheit_np = daten_np * 1.8 + 32` aus. Geben Sie das resultierende Array aus.

Erwartetes Ergebnis im Terminal:

```
Fahrenheit Werte: [72.5  73.58 71.24 75.2  73.22 74.3  71.78 74.84 71.6  76.1  73.4 ]
```

8.4 Kontrollfragen zu Modul 7

1. Wofür steht die Abkürzung NumPy?
2. Welches zentrale Datenobjekt ist das Herzstück von NumPy?
3. Erklären Sie den Begriff „Vektorisierung“ im Kontext von Datenverarbeitung in Python.

9 Modul 8: Fortgeschrittene NumPy Analyse (Boolesche Maskierung)

In diesem Modul ahmen wir die Datenbereinigung aus Modul 6 nach, aber dieses Mal auf die moderne NumPy-Art, wie sie im Dokument „002_Beispiel Datenstrukturen - Analyse.pdf“ in Listing 3 beschrieben wird.

9.1 Aufgabe 8.1: Daten laden direkt mit NumPy

NumPy kann CSV-Dateien direkt laden. Da unsere CSV eine Text-Überschrift (Header) und gemischte Datentypen hat, lesen wir vereinfacht nur die Temperatur-Werte aus der Liste, die alle CSV-Werte repräsentiert.

Erstellen Sie ein Array mit allen Rohdaten (inklusive Ausreißern):

```
rohdaten_np = np.array([22.5, 23.1, 105.0, 21.8, -5.0, 24.0, 22.9, 110.5, 23.5,
    22.1, 23.8, 22.0, 24.5, 108.2, 23.0])
```

9.2 Aufgabe 8.2: Boolesche Maskierung anwenden

Die *Boolesche Maskierung* erlaubt es uns, Arrays anhand von Bedingungen zu filtern, komplett ohne `if`-Abfragen oder Schleifen.

Erstellen Sie eine Bedingung: `maske_gueltig = (rohdaten_np > 0) & (rohdaten_np <= 50)` *(Hinweis: Bei NumPy verwenden wir das kaufmännische Und & anstelle des Textes `and`.)*

Wenden Sie die Maske an, um ein bereinigtes Array zu erzeugen: `bereinigte_daten_np = rohdaten_np[maske_gueltig]`

Drucken Sie das bereinigte Array.

9.3 Aufgabe 8.3: Statistische Kennzahlen direkt abrufen

Nutzen Sie die eingebauten C-optimierten Funktionen von NumPy, um die Auswertung durchzuführen. Berechnen Sie für die `bereinigte_daten_np`:

- `np.mean()` für den Mittelwert
- `np.median()` für den Median
- `np.std()` für die Standardabweichung

Geben Sie die Werte gerundet (Tipp: `round(wert, 2)`) aus.

Erwartetes Ergebnis im Terminal:

```
NumPy Auswertung:
Mittelwert: 23.11
Median: 23.05
Standardabweichung: 0.89
```

9.4 Kontrollfragen zu Modul 8

1. Warum ist `bereinigte_daten_np = rohdaten_np[rohdaten_np <= 50]` eleganter und schneller als die Methode mit der leeren Liste und der `for`-Schleife aus Modul 6?
2. Worin unterscheiden sich statistisch gesehen der Mittelwert und der Median? Welcher Wert ist anfälliger für nicht-gefilterte Ausreißer?

10 Modul 9: Visualisierung der Daten mit Matplotlib

Zahlen im Terminal sind gut, Diagramme sind besser. Um die Auswirkungen der Ausreißer zu demonstrieren, visualisieren wir die Daten mit `matplotlib`.

10.1 Aufgabe 9.1: Matplotlib installieren und importieren

Installieren Sie das Modul über Ihr Terminal.

Tipp zur Installation:
`pip install matplotlib`
Import in Python: `import matplotlib.pyplot as plt`

10.2 Aufgabe 9.2: Ein einfaches Liniendiagramm

Erstellen Sie einen einfachen Plot für die **Rohdaten** (`rohdaten_np`), um die Ausreißer visuell erkennbar zu machen. Nutzen Sie `plt.plot(rohdaten_np, marker='o', color='red')`. Fügen Sie einen Titel mit `plt.title()` sowie Achsenbeschriftungen mit `plt.xlabel()` und `plt.ylabel()` hinzu. Zeigen Sie das Diagramm mit `plt.show()` an.

10.3 Aufgabe 9.3: Subplots (Vorher / Nachher Vergleich)

Wir wollen das Dokument `002_Beispiel Datenstrukturen - Analyse.pdf` exakt nachbauen (Listing 4). Erstellen Sie zwei Diagramme nebeneinander (Subplots).

```
1 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12,5))
```

- Plot 1 (`ax1`): Liniendiagramm der Originaldaten (mit Ausreißern) in Rot.
- Plot 2 (`ax2`): Balkendiagramm (`.bar()`) der `bereinigte_daten_np` in Blau. (Tipp: Die x-Achse ist `range(len(bereinigte_daten_np))`).

Setzen Sie für beide Diagramme Titel und Achsenbeschriftungen. Rufen Sie am Ende `plt.tight_layout()` und `plt.show()` auf.

Erwartetes Ergebnis: Es öffnet sich ein Fenster mit zwei Graphen nebeneinander. Links schlägt die rote Linie extrem nach oben (über 100) und unten (unter 0) aus. Rechts sehen Sie ein gleichmäßiges blaues Balkendiagramm, bei dem sich alle Werte zwischen 20 und 30 einpendeln.

10.4 Kontrollfragen zu Modul 9

1. Was bewirkt der Befehl `plt.subplots(1, 2)`? Was bedeuten die Zahlen 1 und 2?
2. Warum ist es essentiell, Achsenbeschriftungen (`xlabel`, `ylabel`) in einem Diagramm anzugeben?
3. Welche Funktion übernimmt `plt.tight_layout()`?

11 Modul 10: Erweiterte Visualisierung mit Seaborn

`Matplotlib` ist das Arbeitstier, aber `Seaborn` baut darauf auf und erstellt mit extrem wenig Code hochprofessionelle, statistische Grafiken.

11.1 Aufgabe 10.1: Seaborn installieren

Installieren Sie Seaborn.

Tipp zur Installation:
`pip install seaborn`
Import: `import seaborn as sns`

11.2 Aufgabe 10.2: Der Boxplot zur Ausreißererkennung

In der Statistik ist der sogenannte „Boxplot“ das Standardinstrument, um die Verteilung von Daten darzustellen und Ausreißer visuell und mathematisch korrekt auszuweisen.

Konfigurieren Sie Seaborn, um ein schönes Hintergrunddesign zu nutzen: `sns.set_theme(style="whitegrid`

Erstellen Sie eine neue Abbildung (Figure). Übergeben Sie Ihr Array mit den Rohdaten (`rohdaten_np`) an die Boxplot-Funktion: `sns.boxplot(y=rohdaten_np, color="cyan")`

Setzen Sie einen Titel: „Verteilung der Sensordaten inkl. Ausreißer“ und zeigen Sie den Plot mit `plt.show()` an.

Erwartetes Ergebnis: Sie sehen einen Kasten (die Box), der den Großteil der Daten zwischen ≈ 22 und 24 repräsentiert. Weit oberhalb (bei 105 und 110) und weit unterhalb (bei -5) der eigentlichen Linie (den Whisker) befinden sich einzelne Punkte (Diamonds). Dies sind die von Seaborn automatisch als statistische Ausreißer identifizierten Werte.

11.3 Aufgabe 10.3: Der finale Report (Kombination aller Techniken)

Fassen Sie Ihren gesamten Code aus den Modulen 7 bis 10 in einer sauberen, aufeinanderfolgenden Struktur zusammen:

1. Daten einlesen.
2. Boolesche Maske erstellen und bereinigten Datensatz generieren.
3. Konsole: Print-Anweisungen für Mittelwert und Median ausgeben.
4. Visualisierung: Ein Matplotlib Subplot-Fenster generieren, das als linken Plot den Seaborn Boxplot der Originaldaten und als rechten Plot ein Seaborn Histogramm (`sns.histplot`) der bereinigten Daten anzeigt.

11.4 Kontrollfragen zu Modul 10

1. Auf welcher Bibliothek basiert Seaborn unter der Haube? (Lesen Sie dazu den Vorlesungstext zu NumPy).
2. Was zeigt ein Boxplot (auch Schachteldiagramm genannt) grundlegend an?
3. Reflektieren Sie: Sie haben nun den Weg von einzelnen Variablen und fehleranfälligen `for`-Schleifen bis hin zu hocheffizienten, vektorisierten Arrays und statistischen Plotting-Bibliotheken zurückgelegt. Warum startet man in der Informatik überhaupt mit einfachen Listen und Schleifen, wenn NumPy alles viel schneller kann?

Herzlichen Glückwunsch!

Sie haben das Grundlagen-Projekt erfolgreich abgeschlossen und einen vollständigen analytischen Pipeline-Prozess in Python implementiert.